

LeafSegmenter: An Interactive, GPU-Optional Desktop Application for Plant Image Segmentation and Phenotyping Using the Segment Anything Model 2

Nirwan Tandukar¹

¹[Your Institution / Department]

[your.email@institution.edu]

July 5, 2026

Abstract

Accurate segmentation of plant structures in field and greenhouse imagery is a prerequisite for high-throughput, automated phenotyping. Existing tools either demand substantial computational resources, require command-line expertise, or are limited to a single segmentation paradigm. We present **LeafSegmenter**, an open-source, cross-platform desktop application that combines the state-of-the-art Segment Anything Model 2 (SAM2) with a suite of interactive mask-editing, image-enhancement, and model-training tools accessible to non-expert users on standard laptop hardware. The application supports both GPU-accelerated inference (CUDA and Apple MPS) and fully CPU-based operation, making it accessible without high-performance computing infrastructure. Key capabilities include an interactive colour-based mask filter, geometric and species-specific leaf-completion templates, a leaf-unfolding module based on mirror reflection across the detected fold line, an undo history system, and a training pipeline that allows users to distil their

15 curated SAM2 masks into a lightweight U-Net model for rapid, automatic deployment
16 on new images. Comprehensive phenotyping metrics (area, length, width, shape
17 descriptors, colour statistics, and vegetation indices) are exported as structured CSV
18 files. LeafSegmenter lowers the barrier to entry for plant phenotyping research while
19 remaining extensible to heavier architectures and novel species.

20 **Keywords:** plant phenotyping, image segmentation, Segment Anything Model, vision
21 transformer, interactive tools, leaf morphology, field imaging.

1 Introduction

Plant phenotyping — the quantitative measurement of plant morphology, colour, and structural traits — is central to modern crop improvement programmes and fundamental plant biology research (??). The increasing accessibility of digital cameras and unmanned aerial vehicles has generated vast quantities of plant imagery from both field trials and controlled greenhouse environments, yet the bottleneck in extracting meaningful measurements has shifted from image acquisition to accurate, scalable *segmentation*: the process of delineating individual plant structures from their backgrounds and from one another (?).

1.1 Segmentation in Plant Science

Instance segmentation — assigning a distinct mask to each individual object (leaf, stem, fruit) in an image — remains one of the most challenging tasks in agricultural computer vision. Leaves overlap, lighting conditions vary dramatically between field and greenhouse settings, colour contrasts between plant tissue and background are often subtle, and plant morphology varies substantially across growth stages, genotypes, and stress conditions (??). Early approaches relied on classical colour thresholding and morphological operations (?), which perform well only under tightly controlled conditions. The advent of deep learning, and convolutional neural networks in particular, substantially advanced segmentation quality (??), but these approaches require large annotated datasets and significant computational resources, limiting their adoption outside well-funded research groups.

1.2 Foundation Models and the Segment Anything Model

A transformative development in computer vision has been the emergence of large-scale *foundation models*: neural networks pre-trained on internet-scale datasets that generalise remarkably well to novel domains with little or no additional fine-tuning (?). For image segmentation, Meta AI Research released the Segment Anything Model (SAM) in 2023

(?), followed by the substantially improved SAM 2 in 2024 (?).

SAM 2 employs a hierarchical Vision Transformer (ViT) backbone (?) coupled with a streaming memory module originally designed for video object segmentation. The ViT backbone divides the input image into fixed-size patches, encodes them into high-dimensional token representations, and processes them through multi-head self-attention layers that capture long-range spatial dependencies without the inductive bias of convolutions (??). The result is a powerful, promptable segmentation model: given point, box, or mask prompts from a user, SAM 2 generates high-quality instance masks for arbitrary object categories. Critically, SAM 2 operates in a zero-shot regime — it requires no training on plant-specific data to segment leaves, stems, or fruits (?).

Several studies have demonstrated the effectiveness of SAM and SAM 2 for agricultural applications, including crop row detection, fruit counting, and leaf instance segmentation (??). However, direct use of SAM 2 in a research workflow requires Python programming knowledge, access to the model weights (up to 2.4 GB), and familiarity with its API, presenting a significant barrier for plant biologists without computational training.

1.3 Vision Transformers and Lightweight Alternatives

While full ViT-based models such as SAM 2 deliver state-of-the-art segmentation quality, their inference cost can be prohibitive on standard laptop hardware. Encoder-decoder architectures based on U-Net (?) paired with lightweight ImageNet-pre-trained convolutional encoders (ResNet (?), EfficientNet (?)) offer a practical trade-off: substantially faster inference with moderate accuracy loss. MobileSAM (?), a distilled variant of SAM trained to replicate its image encoder using a lightweight ViT-Tiny backbone, achieves segmentation quality close to the full SAM while reducing model size by over 60-fold, making deployment on CPU feasible.

The knowledge-distillation paradigm — training a compact *student* model to reproduce the outputs of a powerful *teacher* model (?) — forms the conceptual basis of the training pipeline in the present work: users collect high-quality masks generated by SAM 2, then use those masks to supervise training of a lightweight U-Net for rapid, class-specific, fully

automatic deployment.

1.4 Motivation and Scope

Despite the power of modern segmentation models, accessible, interactive tooling for plant phenotyping remains scarce. Tools such as LabelMe (?), CVAT, and Roboflow provide general-purpose annotation interfaces but lack plant-specific enhancement pipelines, phenotype measurement modules, or integrated model-training workflows. PlantCV (?) offers a comprehensive plant image analysis library but operates through Python scripts rather than a graphical interface. iMeasure and similar tools support morphometric measurements but not deep-learning-based segmentation.

We therefore developed **LeafSegmenter**, an open-source desktop application that addresses all of the following requirements simultaneously:

1. GPU-optional operation: full functionality on laptop CPU, with optional acceleration via CUDA (NVIDIA) or Apple MPS (Apple Silicon).
2. Interactive, SAM2-based segmentation with no coding required.
3. A rich suite of image-enhancement, mask-editing, and phenotype-measurement tools designed specifically for plant imagery.
4. An integrated training pipeline enabling users to convert curated SAM 2 masks into a deployable lightweight model.
5. Novel tools for geometric leaf completion and leaf-unfolding reconstruction, extending the utility of partial or occluded masks.

2 Materials and Methods

2.1 Application Architecture

LeafSegmenter is implemented in Python 3.11 using the `tkinter` toolkit for the graphical interface. The application is structured as a modular package (`app/`) comprising seven

100 functional modules:

- 101 • `gui_app.py` — main application class, layout, event handling
- 102 • `image_processing.py` — pure image enhancement functions
- 103 • `mask_utils.py` — mask mathematics and shape completion geometry
- 104 • `phenotyping.py` — phenotype measurement and CSV export
- 105 • `sam2_utils.py` — SAM 2 model loading and Hydra configuration
- 106 • `color_filter.py` — interactive colour-based mask filtering
- 107 • `tab_leaf_completion.py`, `tab_leaf_unfolding.py`, `tab_train.py` — training
- 108 panel tab modules

109 The modular design ensures that computationally intensive functions (image enhance-
110 ment, mask geometry, phenotype measurement) are decoupled from the GUI layer and
111 can be invoked independently for batch processing or unit testing.

112 **2.2 Image Loading and Preprocessing**

113 The application accepts TIFF, PNG, JPEG, and BMP images. A built-in TIF converter
114 supports conversion of raw field-acquisition formats to standard 8-bit PNG prior to
115 segmentation. Images are loaded as RGB `uint8` NumPy arrays via Pillow and OpenCV.

116 A rotation module supports arbitrary angle rotation via a dial widget with preset
117 shortcuts (45° , 90° , 180° , 270° , -45° , -90°), allowing correct orientation of images acquired
118 at non-standard angles before segmentation.

119 **2.3 Image Enhancement Pipeline**

120 A dedicated enhancement panel provides a sequential processing pipeline designed for
121 plant imagery. The pipeline is configurable through the GUI and includes the following
122 operations:

2.3.1 Green-Channel CLAHE Enhancement

The primary plant-specific enhancement applies Contrast Limited Adaptive Histogram Equalisation (CLAHE) (?) exclusively to the value channel (V) of pixels identified as green by HSV thresholding. This selectively improves contrast within leaf tissue while leaving background regions unaffected, improving the signal-to-background ratio for SAM 2. A bilateral filter (?) is subsequently applied to reduce noise while preserving leaf edges, followed by Sobel edge blending (?) to further accentuate boundary regions. An unsharp masking step (?) completes the standard plant pipeline.

2.3.2 Vegetation Indices

The application computes five widely used vegetation indices from the RGB channels, which may be used either as visual enhancement overlays or as input features for downstream analysis:

- **ExG** (Excess Green): $\text{ExG} = 2G - R - B$ (?)
- **ExGR**: $\text{ExGR} = \text{ExG} - 1.4R - G$
- **GRVI** (Green-Red Vegetation Index): $\text{GRVI} = (G - R)/(G + R)$ (?)
- **VARI** (Visible Atmospherically Resistant Index): $\text{VARI} = (G - R)/(G + R - B)$ (?)
- **GLI** (Green Leaf Index): $\text{GLI} = (2G - R - B)/(2G + R + B)$ (?)

2.3.3 Advanced Enhancement Options

Additional enhancement methods are available for challenging imaging conditions:

- **Retinex** (single- and multi-scale) (?) for illumination normalisation under variable field lighting.
- **Morphological top-hat transform** (?) for uneven background illumination correction.
- **Non-local means denoising** (?) for high-noise images.

- **Guided filtering** (?) for edge-preserving smoothing using the green channel as guide.
- **LAB colour-space enhancement, grey-world and max-white white balance, Difference of Gaussians, local contrast normalisation, adaptive gamma correction, and shadow/highlight recovery.**
- **Background whitening:** HSV-based detection of bright, desaturated background pixels (e.g. white imaging trays) with morphological cleanup, replacing them with white to simplify foreground segmentation.

2.4 SAM2-Based Segmentation

LeafSegmenter integrates SAM 2 (?) via the official Python API. The user specifies a SAM 2 checkpoint (Hiera-Small, Hiera-Large, etc. or a bundled .pt file) and the target compute device (CPU, CUDA, or Apple MPS). Segmentation is triggered on the current image after enhancement, producing a set of instance masks via `SAM2AutomaticMaskGenerator` with fully configurable hyperparameters exposed in the GUI:

- `points_per_side`: grid density of prompt points.
- `pred_iou_thresh`: predicted IoU quality threshold.
- `stability_score_thresh`: mask stability rejection threshold.
- `crop_n_layers`: number of multi-scale crop passes.
- `crop_overlap_ratio`: overlap between adjacent crops.
- `box_nms_thresh`: duplicate suppression threshold.
- `min_mask_region_area`: minimum mask area in pixels.
- `use_m2m`: mask-to-mask refinement flag.

Hover tooltips in the GUI provide plain-language explanations of each parameter. After generation, masks are deduplicated by Intersection-over-Union ($\text{IoU} > 0.80$) and split

into single-connected-component instances. Batch segmentation processes an entire folder, caching results in memory for subsequent review and export.

2.5 Mask Editing Tools

A dedicated Masks panel provides interactive editing operations:

- **Delete:** remove selected masks from the set.
- **Combine:** union of two or more selected masks into one.
- **Refine:** re-run SAM 2 within the bounding box of a selected mask using denser prompting for improved boundary accuracy.
- **Edit boundary:** an interactive contour editor allowing point-by-point drag adjustment of mask boundaries, with add-point (Ctrl+click) and delete-point (Shift+click) operations and optional Gaussian smoothing.
- **Split:** decompose multi-blob masks into individual connected-component masks.

An undo history system (up to 50 states) captures the full mask list before each destructive operation, enabling recovery via Ctrl+Z (or Cmd+Z on macOS).

2.6 Colour-Based Mask Filtering

A colour filter panel provides interactive mask selection based on pixel colour statistics. An interactive two-dimensional Hue $\times$ Saturation picker renders the full HSV colour space as a gradient. Users drag rectangles on the picker to define one or more colour ranges; these are combined with OR logic. An eyedropper tool samples a pixel directly from the preview canvas, placing a crosshair marker on the picker at the sampled position and automatically generating a selection rectangle with a sensible tolerance. A separate brightness (Value) range control applies globally across all selections. The filter operates on the mean HSV colour of each mask's constituent pixels, allowing rapid removal of soil, background, or specular highlight masks without manual selection.

2.7 Leaf Shape Completion

Many plant images contain partially occluded or truncated leaves. To support reconstruction of the complete leaf area for morphometric analysis, LeafSegmenter includes a leaf completion module offering two complementary shape galleries:

2.7.1 Geometric Shape Gallery

Eighteen botanical leaf forms are available (Orbicular, Elliptical, Ovate, Obovate, Cordate, Reniform, Lanceolate, Oblanceolate, Oblong, Linear, Ensiform, Cuneate, Spathulate, Deltoid, Rhomboid, Sagittate, Hastate, and a general Ellipse), each rendered as a silhouette thumbnail using parametric polygon definitions.

2.7.2 Crop Species Gallery

Eighteen crop-specific silhouettes are provided, traced from documented leaf morphology: Maize, Wheat, Sorghum, Rice, Barley (monocots with narrow strap leaves), Arabidopsis (rosette form), Tomato and Potato (pinnate compound), Soybean (trifoliate), Sunflower, Tobacco, Cassava (palmate), Cotton, Grape, Common Bean, Cucumber, Pepper, and Lettuce.

2.7.3 Alignment Algorithm

Given a selected mask and shape template, the completion algorithm performs three sequential alignment steps:

1. **Centroid alignment:** the shape template is translated so that its centre coincides with the pixel centre of mass (centroid) of the mask.
2. **PCA orientation:** Principal Component Analysis is applied to the mask pixel coordinates. The major eigenvector defines the leaf's primary axis; the template is rotated to align its major axis with this direction.
3. **Scale fitting:** the template axes are scaled to match the PCA-derived major and minor axis lengths of the mask.

The aligned template is rasterised at full image resolution and merged with the original mask (preserving all original pixels). Fine adjustments to scale, position (drag), and rotation angle are available interactively before the completed mask is committed.

2.8 Leaf Unfolding Reconstruction

Folded leaves are common in greenhouse and field imagery, particularly in maize, sorghum, and wheat. A dedicated Leaf Unfolding module reconstructs the complete leaf by mirroring the folded portion across the detected fold line.

The fold line is detected automatically as the junction boundary between two selected masks (representing the two halves of the folded leaf). Both masks are dilated morphologically and their intersection defines the junction zone. Singular Value Decomposition (SVD) is applied to the junction pixel coordinates, with the first right singular vector defining the fold line direction. Each pixel $\mathbf{p}$ of the folded mask is then reflected as:

$$\mathbf{p}' = \mathbf{p} - 2[(\mathbf{p} - \boldsymbol{\mu}) \cdot \hat{\mathbf{n}}] \hat{\mathbf{n}} \quad (1)$$

where $\boldsymbol{\mu}$ is a point on the fold line and $\hat{\mathbf{n}}$ is the unit normal to the fold line direction. The reflected mask is morphologically closed to fill rounding artefacts, and the corresponding image pixels are mapped to their reflected positions. The user may fine-tune the fold line angle ($\pm 5^\circ$ increments) and shift the mirrored piece interactively before applying.

2.9 Phenotype Measurement and Export

For each mask (or set of selected masks), the following measurements are computed and exported as CSV files:

2.9.1 Morphometric Descriptors

- **Area** (px²)
- **Length and width:** PCA major/minor axis lengths; deskewed row-span length and 95th-percentile row width.

- **Perimeter, convex hull area and perimeter**
- **Solidity:** mask area / convex hull area
- **Circularity:** $4\pi A/P^2$
- **Extent:** mask area / bounding box area
- **Equivalent diameter:** diameter of a circle of equal area
- **Connected components:** number of distinct blobs

2.9.2 Colour Descriptors

- Per-channel (R, G, B, H, S, V) mean, median, sum, and standard deviation within the mask
- HSV channel variances
- Vegetation index means: ExG, ExR, ExGR, GLI, green pixel fraction

Both *individual* (per-segment) and *joint* (aggregated across a selection) CSV exports are supported. Masks may also be saved as full-size binary PNG files, bounding-box-cropped binary masks, and RGBA transparent-background crops.

2.10 Custom Model Training

To enable rapid, class-specific segmentation without SAM 2 at inference time, LeafSegmenter includes an integrated training pipeline.

2.10.1 Data Collection

Users collect training examples through normal SAM 2-assisted segmentation: after segmenting an image, selected masks are added to a structured dataset (images in **images/**, binary masks in **masks/**) via a single button press. Images may also be marked as negative examples (no target present).

2.10.2 Model Architecture

The default architecture is a U-Net (?) with a ResNet-18 (?) encoder pre-trained on ImageNet (?), implemented via the `segmentation-models-pytorch` library (?). A lightweight alternative (`unet_small`) omits the pre-trained encoder for resource-constrained scenarios. The binary segmentation decoder produces a single-channel output mask.

2.10.3 Training Procedure

Training uses the Adam optimiser (?) with a configurable learning rate (default 10^{-5}), batch size, image size, device, and number of gradient steps. A weighted binary cross-entropy loss accounts for class imbalance between foreground and background pixels. Training can be resumed from an existing checkpoint. All hyperparameters are exposed in the GUI without requiring code modification.

2.10.4 Inference

The trained model accepts arbitrary-resolution images, applies the same enhancement pipeline as SAM 2 segmentation, and produces instance masks via connected-component analysis of the thresholded output probability map. A sliding-window (tile-based) inference mode handles large images by processing overlapping tiles and merging predictions. An optional colour-guided tile scanning mode restricts tile processing to regions containing plant-coloured pixels, substantially reducing inference time on large field images.

Acknowledgements

[Your acknowledgements here.]